Diffusion Models II

Lecture 8: Score-based generative modeling

Xingche Guo

STAT 5095 University of Connecticut · Fall 2026

The score function

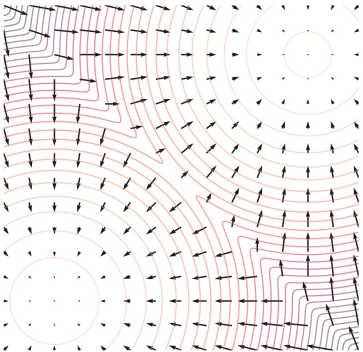
We observe $x_0^{(1)}, \ldots, x_0^{(N)} \sim q_0$ and want to generate new data.

A density tells us **how much probability** lies near x.

Its (Stein) **score function** tells us how the log-density changes as we move x:

$$\boxed{s(x) = \nabla_x \log q_0(x) \in \mathbb{R}^D.}$$

- Learn this vector field from data.
- Use it to guide a random sampling process.



Contours: density. Arrows: score.

Why learn the score?

Suppose we model the data density as

$$p_\theta(x) = \frac{\exp\{-f_\theta(x)\}}{Z_\theta}, \qquad Z_\theta = \int \exp\{-f_\theta(u)\}\, du.$$

Evaluating Z_θ can be difficult in high dimensions.

1. The score removes the normalizing constant. Since Z_θ does not depend on x,

$$\boxed{\nabla_x \log p_\theta(x) = -\nabla_x f_\theta(x) - \underbrace{\nabla_x \log Z_\theta}_{0}.}$$

2. The exact score uniquely identifies the density.
For smooth densities p and q on a connected domain. Suppose for almost every $x > 0$,

$$\nabla_x \log p(x) = \nabla_x \log q(x) \implies \nabla_x \log \frac{p(x)}{q(x)} = 0 \implies p(x) = c\, q(x).$$

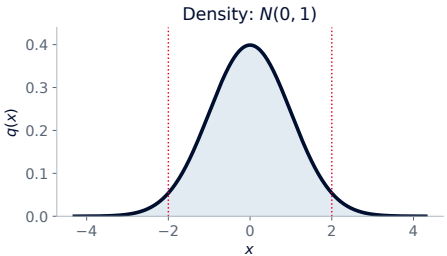
Both densities integrate to one, so $c = 1$ and therefore $p = q$.

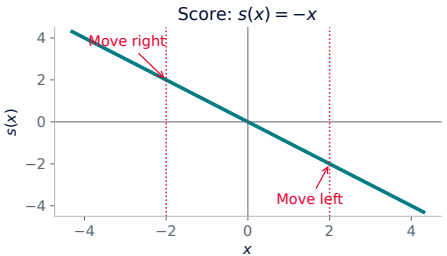
This motivates learning the score directly from data, then using it to guide sampling.

A Gaussian example

For $q(x) = \mathcal{N}(\mu, \tau^2)$,

$$s(x) = \frac{d}{dx} \log q(x) = -\frac{x - \mu}{\tau^2}.$$





- The sign gives a direction; the magnitude reflects the local log-density slope.
- For $\mathcal{N}(\mu, \Sigma)$ in $\mathbb{R}^D$: $s(x) = -\Sigma^{-1}(x - \mu)$.

Sampling with Langevin dynamics

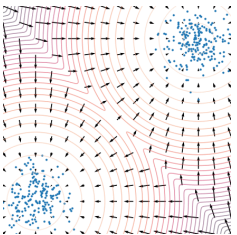
How can we sample from q_0 if we know only its score?

Use **Langevin dynamics**, with $s(x) = \nabla_x \log q_0(x)$:

$$x^{(k+1)} = x^{(k)} + \underbrace{\frac{\alpha}{2} s(x^{(k)})}_{\text{score drift}} + \underbrace{\sqrt{\alpha}\, z_k}_{\text{random exploration}},$$

with $z_k \overset{\text{i.i.d.}}{\sim} \mathcal{N}(0, I_D)$.

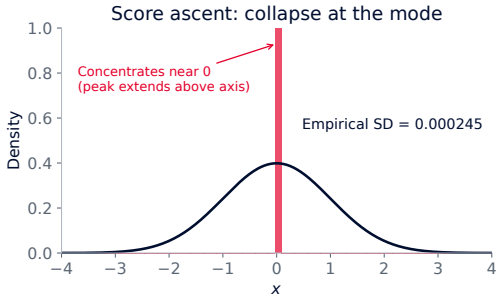
- The score drift favors higher-density regions.
- The noise maintains a spread of samples.
- Small steps and sufficient mixing give approximate samples from q_0 (i.e., $x^{(T)} \overset{\text{approx}}{\sim} q_0$).

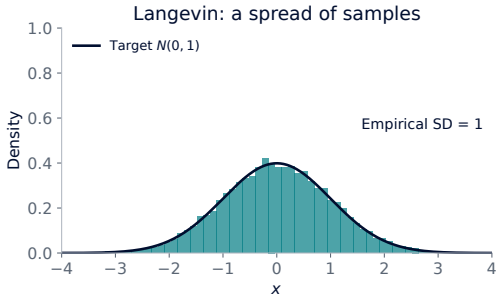


Open animation in browser

Why add randomness?

For $q = \mathcal{N}(0, 1)$, $s(x) = -x$. Compare updates **without** and **with** added noise.





Score ascent: samples collapse toward the mode $x = 0$.

Langevin sampling: samples approximate the target distribution q.

Open animation in browser

Simulation: 10,000 chains, 240 updates, $\alpha = 0.07$; shared initial distribution $\mathcal{N}(3, 1.25^2)$.

Score matching

Let $X \sim q_0$ and we model the data density as $p_\theta(x)$. Let $s_\theta = \nabla_x \log p_\theta(x) : \mathbb{R}^D \to \mathbb{R}^D$ be a score network. The **score matching** idea is to minimize the **Fisher divergence**

$$\boxed{\mathcal{L}_{\mathrm{F}}(\theta) = \mathbb{E}_{X \sim q_0}\left[\|s_\theta(X) - \nabla_x \log q_0(X)\|^2\right].}$$

This objective is inaccessible: we observe samples from q_0, but their true scores $\nabla_x \log q_0(X)$ are unknown.

Expand the square and use $q_0(x)\nabla_x \log q_0(x) = \nabla_x q_0(x)$:

$$\mathcal{L}_{\mathrm{F}}(\theta) = \mathbb{E}_{q_0}\|s_\theta(X)\|^2 - 2\,\mathbb{E}_{q_0}\left[s_\theta(X)^\top \nabla_x \log q_0(X)\right] + \underbrace{\mathbb{E}_{q_0}\|\nabla_x \log q_0(X)\|^2}_{\text{independent of } \theta}$$

$$= \mathbb{E}_{q_0}\|s_\theta(X)\|^2 - 2\int s_\theta(x)^\top \nabla_x q_0(x)\,dx + \underbrace{\mathbb{E}_{q_0}\|\nabla_x \log q_0(X)\|^2}_{\text{independent of } \theta}.$$

Eliminating the unknown score

Assume smoothness, integrability, and vanishing boundary terms. Integration by parts (IBP) gives

$$\begin{aligned}
\int s_\theta(x)^\top \nabla_x q_0(x)\, dx &= \sum_{j=1}^{D} \int s_{\theta,j}(x) \frac{\partial q_0(x)}{\partial x_j}\, dx \\
&\overset{\text{IBP}}{=} -\sum_{j=1}^{D} \int q_0(x) \frac{\partial s_{\theta,j}(x)}{\partial x_j}\, dx \\
&= -\mathbb{E}_{q_0}\left[\sum_{j=1}^{D} \frac{\partial s_{\theta,j}(X)}{\partial x_j}\right] \\
&= -\mathbb{E}_{q_0}\left[\operatorname{tr}(\nabla_x s_\theta(X))\right].
\end{aligned}$$

Here $\nabla_x s_\theta(x) \in \mathbb{R}^{D\times D}$ is the network's Jacobian matrix.

An accessible but costly objective

The score-matching objective is

$$\mathcal{L}_{\mathrm{SM}}(\theta) = \mathbb{E}_{X \sim q_0}\left[2\,\mathrm{tr}(\nabla_x s_\theta(X)) + \|s_\theta(X)\|^2\right].$$

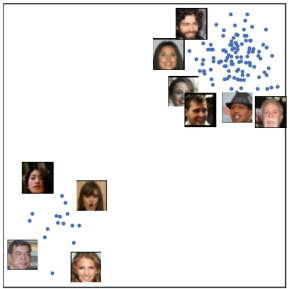
We can estimate this expectation by averaging over the observed data. Neither $q_0(x)$ nor its score needs to be evaluated.

The remaining difficulty is the Jacobian trace:

$$\mathrm{tr}(\nabla_x s_\theta(x)) = \sum_{j=1}^{D} \frac{\partial s_{\theta,j}(x)}{\partial x_j}.$$

- For a general network, evaluating the exact trace can require D reverse-mode passes per observation.
- Training also differentiates these input derivatives with respect to θ.

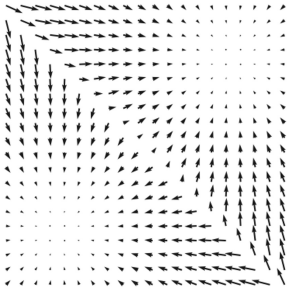
Score matching Langevin dynamics (SMLD)



Data samples

$\{\mathbf{x}_1, \mathbf{x}_2, \cdots, \mathbf{x}_N\} \overset{\text{i.i.d.}}{\sim} p(\mathbf{x})$

score matching



Scores

$\mathbf{s}_\theta(\mathbf{x}) \approx \nabla_{\mathbf{x}} \log p(\mathbf{x})$

Langevin dynamics



New samples

Why does this simple recipe struggle for images?

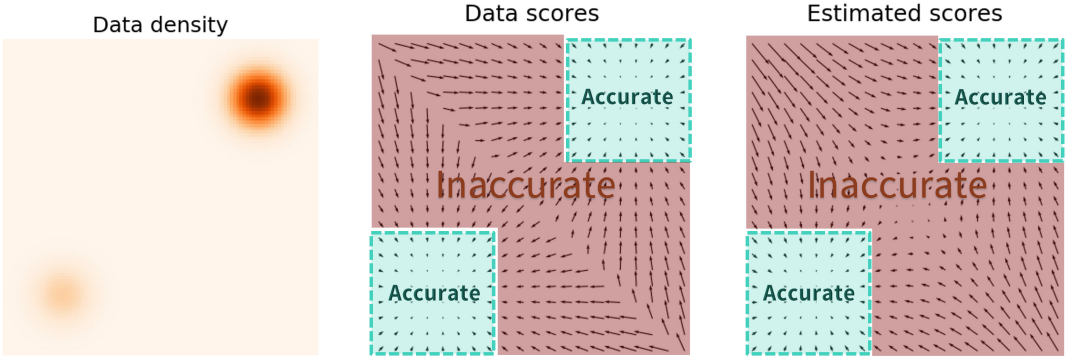
Why naive SMLD struggles

- **High computational cost.** The naive obective function is costly to compute.
- **The score may not be well defined.** Images concentrate near a **lower-dimensional manifold**. If the data lie *exactly* on this manifold, an ordinary density on $\mathbb{R}^D$ does not exist, so neither does the ambient score $\nabla_x \log q_0(x)$.
- **Unreliable scores in low-density regions.** Training data provide little information about these regions, yet randomly initialized samples may start there.
- **Slow mixing between modes.** Langevin dynamics may remain near one mode for a long time, struggling to cross low-density gaps and explore other modes.

Scores in low-density regions

Score matching minimizes the squared error between the estimated and true scores, **weighted by the data density** $q_0(x)$.

Errors in **low-density regions** contribute little to the objective, so the learned score may be inaccurate there.

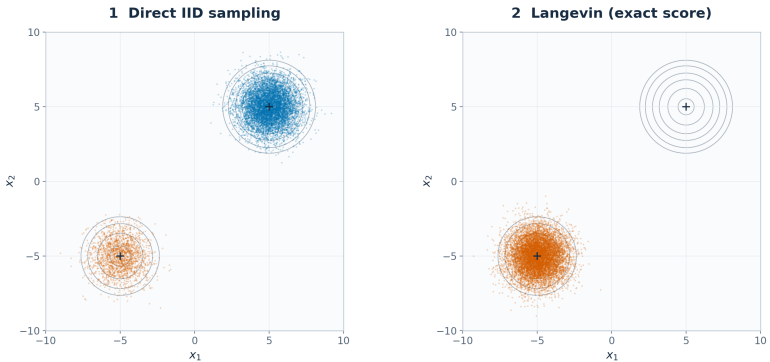


Even exact scores can mix slowly

For a mixture with **well-separated modes**, Langevin dynamics can explore one mode while struggling to reach the other. Here, chains initialized near one mode:

$$q_0(x) = 0.2\,\mathcal{N}(x; \mu_1, I_2) + 0.8\,\mathcal{N}(x; \mu_2, I_2)$$

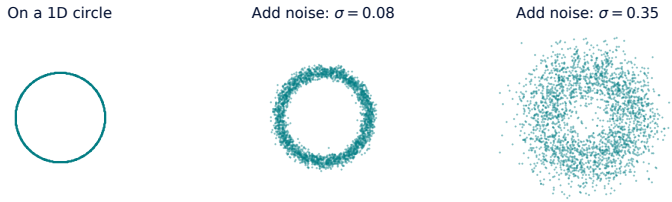
$\mu_1 = (-5, -5)$, $\mu_2 = (5, 5)$ Langevin initialization: $X_0 \sim \mathcal{N}(\mu_1, 0.3^2 I_2)$



Open animation in browser

Noise-perturbed distribution

Think of replacing each clean observation by a **Gaussian cloud of nearby points**. This creates smoother distributions on which to learn and sample.



- **Make the score well-defined:** Gaussian smoothing yields a smooth, strictly positive density, even when the clean distribution lies on a lower-dimensional manifold.
- **Broaden data coverage:** Noisy observations cover wider neighborhoods around the clean data, allowing the score to be learned beyond the original data support.
- **Facilitate exploration:** At higher noise levels, separated clusters overlap more, which can make movement between modes easier.

Denoising score matching

Fix a noise level $\sigma > 0$. Generate a noisy observation using a known corruption distribution:

$$x \sim q_0, \qquad \tilde{x} \mid x \sim q_\sigma(\cdot \mid x).$$

The marginal density of noisy observations is

$$q_\sigma(\tilde{x}) = \int q_0(x)\, q_\sigma(\tilde{x} \mid x)\, \mathrm{d}x.$$

We want to learn the score of this **noise-perturbed distribution**:

$$\boxed{\mathcal{L}_{\mathrm{DSM}}(\theta) = \mathbb{E}_{\tilde{x} \sim q_\sigma}\left[\left\| s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}) \right\|_2^2\right].}$$

Difficulty: the marginal score is unknown.

Key idea: use the known conditional score $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)$ to construct an equivalent objective.

Find the equivalent objective

Writing the expectation as an integral and expanding the square,

$$\begin{aligned}\mathcal{L}_{\mathrm{DSM}}(\theta) &= \int q_\sigma(\tilde{x}) \left\| \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}) - s_\theta(\tilde{x}) \right\|_2^2 \, \mathrm{d}\tilde{x} \\ &= \underbrace{\int q_\sigma(\tilde{x}) \left\| s_\theta(\tilde{x}) \right\|_2^2 \, \mathrm{d}\tilde{x}}_{A_\theta} - 2 \underbrace{\int q_\sigma(\tilde{x}) \left[\nabla_{\tilde{x}} \log q_\sigma(\tilde{x}) \right]^\top s_\theta(\tilde{x}) \, \mathrm{d}\tilde{x}}_{B_\theta} + C_1.\end{aligned}$$

Here,

$$C_1 = \int q_\sigma(\tilde{x}) \left\| \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}) \right\|_2^2 \, \mathrm{d}\tilde{x}$$

does not depend on θ.
Meanwhile

$$\begin{aligned}A_\theta &= \iint q_0(x) q_\sigma(\tilde{x} \mid x) \left\| s_\theta(\tilde{x}) \right\|_2^2 \, \mathrm{d}\tilde{x} dx \\ &= E_{x \sim q_0, \tilde{x} \sim q_\sigma(\cdot \mid x)} \left[\left\| s_\theta(\tilde{x}) \right\|_2^2 \right].\end{aligned}$$

Find the equivalent objective

Use $q_\sigma \nabla \log q_\sigma = \nabla q_\sigma$ and the definition of the noisy marginal:

$$
\begin{aligned}
B_\theta &= \int \left[\nabla_{\tilde{x}} q_\sigma(\tilde{x})\right]^\top s_\theta(\tilde{x}) \, \mathrm{d}\tilde{x} \\
&= \int \left[\nabla_{\tilde{x}} \int q_0(x) q_\sigma(\tilde{x} \mid x) \, \mathrm{d}x\right]^\top s_\theta(\tilde{x}) \, \mathrm{d}\tilde{x} \\
&= \int \left[\int q_0(x) \nabla_{\tilde{x}} q_\sigma(\tilde{x} \mid x) \, \mathrm{d}x\right]^\top s_\theta(\tilde{x}) \, \mathrm{d}\tilde{x} \\
&= \iint q_0(x) q_\sigma(\tilde{x} \mid x) \left[\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\right]^\top s_\theta(\tilde{x}) \, \mathrm{d}x \, \mathrm{d}\tilde{x} \\
&= \mathbb{E}_{x \sim q_0, \tilde{x} \sim q_\sigma(\cdot \mid x)} \left[\left[\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\right]^\top s_\theta(\tilde{x})\right].
\end{aligned}
$$

The cross term now involves the **known corruption kernel**.

Find the equivalent objective

For $x \sim q_0$ and $\tilde{x} \mid x \sim q_\sigma(\cdot \mid x)$, combining the previous results and completing the square gives

$$\begin{aligned}
\mathcal{L}_{\mathrm{DSM}}(\theta) &= \mathbb{E}_{\tilde{x}\sim q_\sigma}\left[\|s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x})\|_2^2\right] \\
&= A_\theta - 2\,B_\theta + C_1 \\
&= \mathbb{E}_{x\sim q_0,\tilde{x}\sim q_\sigma(\cdot|x)}\left[\|s_\theta(\tilde{x})\|_2^2 - 2\big[\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\big]^\top s_\theta(\tilde{x})\right] + C_1 \\
&= \mathbb{E}_{x\sim q_0,\tilde{x}\sim q_\sigma(\cdot|x)}\left[\|s_\theta(\tilde{x}) - \nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)\|_2^2\right] + C_2.
\end{aligned}$$

Here, C_1 and C_2 do not depend on θ.

We can estimate this objective by first sampling clean data $x \sim q_0$, then drawing a noisy observation $\tilde{x} \sim q_\sigma(\cdot \mid x)$. The conditional score $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x)$ is known from the chosen noise model.

Gaussian denoise score matching

For Gaussian corruption,

$$q_\sigma(\tilde{x} \mid x) = \mathcal{N}(\tilde{x}; x, \sigma^2 I_D), \qquad \tilde{x} = x + \sigma\epsilon, \quad \epsilon \sim \mathcal{N}(0, I_D).$$

The conditional score is available in closed form:

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = -\frac{\tilde{x} - x}{\sigma^2} = -\frac{\epsilon}{\sigma}.$$

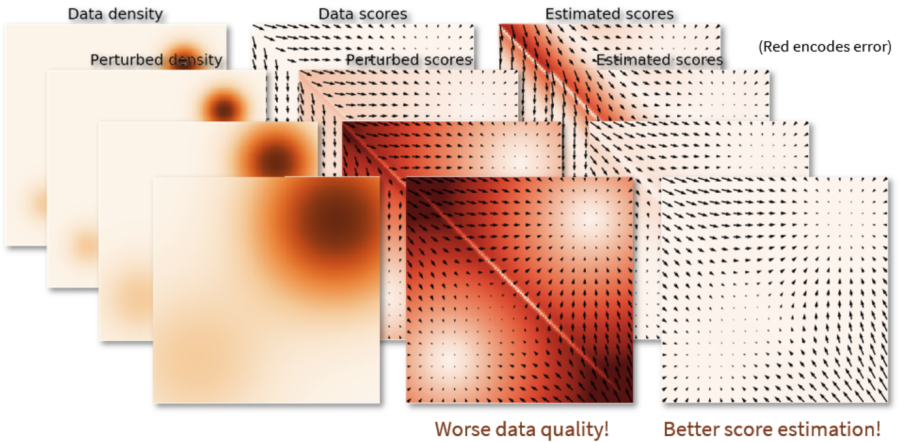
Therefore,

$$\boxed{\mathcal{L}_{\mathrm{DSM}}(\theta) = \mathbb{E}_{x \sim q_0,\ \epsilon \sim \mathcal{N}(0, I_D)} \left[\left\| s_\theta(\tilde{x}) + \frac{\tilde{x} - x}{\sigma^2} \right\|_2^2 \right].}$$

- **Input:** the noisy observation $\tilde{x} = x + \sigma\epsilon$.
- **Target:** $(x - \tilde{x})/\sigma^2$, pointing toward its clean source.
- The network sees only $\tilde{x}$, so squared-error regression averages the conditional targets and learns to approximate the **score of the noise-perturbed distribution**, $\nabla_{\tilde{x}} \log q_\sigma(\tilde{x})$, rather than the original data score $\nabla_x \log q_0(x)$.

Large noise vs small noise

- **Larger noise:** **easier score estimation**, but **far from the target distribution**.
- **Smaller noise:** **close to the target distribution**, but **harder score estimation**.



Noise conditional score networks

Solution: We learn scores across **multiple noise levels** using one shared, noise-conditioned network.

Following Song and Ermon (2019), we use **geometrically spaced noise levels**:

$$\boxed{0 < \sigma_1 < \sigma_2 < \cdots < \sigma_L.}$$

The ratio between adjacent levels is constant:

$$\frac{\sigma_2}{\sigma_1} = \frac{\sigma_3}{\sigma_2} = \cdots = \frac{\sigma_L}{\sigma_{L-1}} = r > 1.$$

Let q_σ denote the density of the perturbed observation

$$\tilde{x} = x + \sigma\epsilon, \qquad x \sim q_0, \quad \epsilon \sim \mathcal{N}(0, I_D).$$

A **noise conditional score network (NCSN)** learns

$$\boxed{\mathbf{s}_\theta(\tilde{x}, \sigma) \approx \nabla_{\tilde{x}} \log q_\sigma(\tilde{x}), \qquad \sigma \in \{\sigma_i\}_{i=1}^L.}$$

The NCSN training objective

Recall: Gaussian perturbations provide a known conditional score

$$q_\sigma(\tilde{x} \mid x) = \mathcal{N}(\tilde{x};\ x, \sigma^2 I), \qquad \nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = -\frac{\tilde{x} - x}{\sigma^2}.$$

For each noise level, use the **denoising score matching loss**:

$$\ell(\theta; \sigma) = \mathbb{E}_{x \sim q_0,\ \tilde{x} \sim q_\sigma(\cdot \mid x)} \left[\left\| s_\theta(\tilde{x}, \sigma) + \frac{\tilde{x} - x}{\sigma^2} \right\|_2^2 \right].$$

Combine all noise levels into one objective:

$$\boxed{\mathcal{L}\Big(\theta; \{\sigma_i\}_{i=1}^L\Big) = \frac{1}{L} \sum_{i=1}^{L} \lambda(\sigma_i)\, \ell(\theta; \sigma_i), \qquad \lambda(\sigma_i) > 0.}$$

With sufficient model capacity, **the minimizer recovers $\nabla_x \log q_{\sigma_i}(x)$ at every noise level**.

Balancing training across noise levels

Recall that $\tilde{x} = x + \sigma\epsilon$, where $\epsilon \sim \mathcal{N}(0, I_D)$. The conditional score target satisfies

$$\nabla_{\tilde{x}} \log q_\sigma(\tilde{x} \mid x) = -\frac{\epsilon}{\sigma}, \qquad \mathbb{E}\left[\left\|\frac{\epsilon}{\sigma}\right\|_2^2\right] = \frac{D}{\sigma^2}.$$

Thus, **smaller noise produces larger targets**. To compensate for this difference in scale, NCSN uses $\boxed{\lambda(\sigma) = \sigma^2}$, giving

$$\mathcal{L}_{\mathrm{NCSN}}(\theta) = \frac{1}{L}\sum_{i=1}^{L} \mathbb{E}_{x\sim q_0,\, \epsilon\sim\mathcal{N}(0,I_D)}\left[\left\|\sigma_i s_\theta(x + \sigma_i\epsilon, \sigma_i) + \epsilon\right\|_2^2\right].$$

Equivalent sampling form. Draw $k \sim \mathrm{Unif}\{1, \ldots, L\}$ independently of x and ϵ:

$$\boxed{\mathcal{L}_{\mathrm{NCSN}}(\theta) = \mathbb{E}_{x,\epsilon,k}\left[\left\|\sigma_k s_\theta(x + \sigma_k\epsilon, \sigma_k) + \epsilon\right\|_2^2\right].}$$

Key interpretation: At every noise level, the scaled score $\sigma_k s_\theta$ is trained against $-\epsilon$, whose expected squared norm is always D.

NCSN training algorithm

Choose geometrically spaced noise levels $0 < \sigma_1 < \cdots < \sigma_L$. Initialize θ, then repeat:

1. Sample a mini-batch $\{x^{(b)}\}_{b=1}^{B}$ from the training data.
2. Independently for each observation, draw a noise level and Gaussian noise:
$$i_b \sim \text{Unif}\{1, \ldots, L\}, \qquad \epsilon^{(b)} \sim \mathcal{N}(0, I_D), \qquad b = 1, \ldots, B.$$
3. Construct the noisy observations:
$$\tilde{x}^{(b)} = x^{(b)} + \sigma_{i_b} \epsilon^{(b)}.$$
4. Take gradient descent step on
$$\boxed{\nabla_\theta \frac{1}{B} \sum_{b=1}^{B} \left\| \sigma_{i_b} s_\theta(\tilde{x}^{(b)}, \sigma_{i_b}) + \epsilon^{(b)} \right\|_2^2.}$$

Key idea: Each observation uses **one randomly sampled noise level**. The mini-batch loss is an unbiased estimate of the objective averaged over all L levels.

Compare NCSN training with DDPM

NCSN: Let $x_0 \sim q_0$. Rewrite the score network as a noise predictor:

$$\tilde{x} = x_0 + \sigma_k \epsilon, \qquad \epsilon_\theta(\tilde{x}, \sigma_k) := -\sigma_k s_\theta(\tilde{x}, \sigma_k).$$

Then the weighted score-matching loss becomes

$$\boxed{\mathcal{L}_{\mathrm{NCSN}}(\theta) = \mathbb{E}_{x_0,k,\epsilon}\left[\|\epsilon_\theta(\tilde{x}, \sigma_k) - \epsilon\|_2^2\right].}$$

DDPM: directly train a noise predictor using

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \epsilon.$$

Its simplified training objective is

$$\boxed{\mathcal{L}_{\mathrm{DDPM}}(\theta) = \mathbb{E}_{x_0,t,\epsilon}\left[\|\epsilon_\theta(x_t, t) - \epsilon\|_2^2\right].}$$

- **Shared idea:** predict the added Gaussian noise from a noisy observation and its noise level.

The forward perturbation process

Like DDPM, we can view increasing noise levels $\sigma_1 < \cdots < \sigma_L$ as a **forward perturbation process** that progressively corrupts clean data. Each conditional marginal can be sampled directly using $\epsilon \sim \mathcal{N}(0, I_D)$:

$$\boxed{\tilde{x}_i = x + \sigma_i \epsilon, \qquad \tilde{x}_i \mid x \sim \mathcal{N}(x, \sigma_i^2 I_D).}$$

A Markov construction with the same marginals. Set $\tilde{x}_0 = x$ and $\sigma_0 = 0$. For $i = 1, 2, \ldots, L$, add independent Gaussian noise:

$$\boxed{\tilde{x}_i = \tilde{x}_{i-1} + \sqrt{\sigma_i^2 - \sigma_{i-1}^2}\, \epsilon_i, \qquad \epsilon_i \overset{\text{iid}}{\sim} \mathcal{N}(0, I_D).}$$

Forward direction: $\tilde{x}_0 = x \longrightarrow \tilde{x}_1 \longrightarrow \cdots \longrightarrow \tilde{x}_L$, with increasing noise.

This Markov chain is **only one way to connect the noise levels**; different joint processes can have the same marginals.

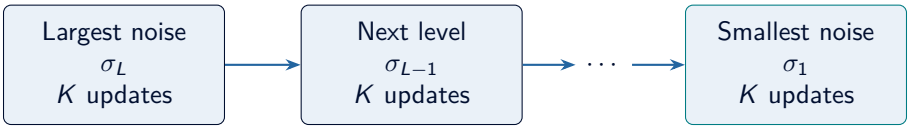
Training samples directly from these marginals, without simulating the entire chain.

Annealed Langevin sampling

After training, we have

$$s_\theta(x, \sigma_i) \approx \nabla_x \log q_{\sigma_i}(x), \qquad 0 < \sigma_1 < \cdots < \sigma_L.$$

Start from a random state and visit the noise levels **from largest to smallest**.



- **Within each block:** fix σ_i and run K Langevin updates, drawing fresh Gaussian noise at every step:

$$x \leftarrow x + \frac{\alpha_i}{2} s_\theta(x, \sigma_i) + \sqrt{\alpha_i}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I_D).$$

- **Large noise encourages exploration:** smoothing can reduce barriers between modes, making it easier to explore different regions of the distribution.
- **Small noise refines the sample:** as noise decreases, the target distribution reveals finer data structure. The final stage targets q_{σ_1}, which approaches q_0 as $\sigma_1 \to 0$.

Sampling algorithm

Inputs: a trained score network s_θ, noise levels $\sigma_1 < \cdots < \sigma_L$, K updates per level, and a base step size $\eta > 0$.

1. **Initialize:** draw $x \sim p_{\text{init}}$ from an easy-to-sample distribution (e.g., Gaussian, uniform).
2. **Anneal from large noise to small noise:** for $i = L, L-1, \ldots, 1$, set

$$\alpha_i = \eta \left(\frac{\sigma_i}{\sigma_1} \right)^2, \qquad \eta > 0.$$

   Thus, the **step size decreases with the noise level**, ending at $\alpha_1 = \eta$.
   At each level, repeat the following update K times, drawing fresh Gaussian noise at every update:

$$\boxed{x \leftarrow x + \frac{\alpha_i}{2} s_\theta(x, \sigma_i) + \sqrt{\alpha_i}\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I_D).}$$

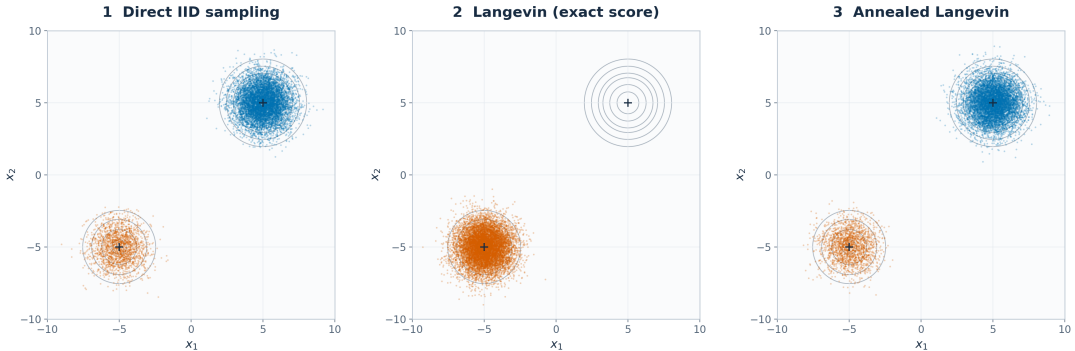
   Carry the final state into the next noise level.
3. **Return:** output x after completing the smallest-noise level σ_1.

A quick demo

Comparing **annealed Langevin sampling** with **ordinary Langevin sampling**:

$$q_0(x) = 0.2\,\mathcal{N}(x; \mu_1, I_2) + 0.8\,\mathcal{N}(x; \mu_2, I_2)$$

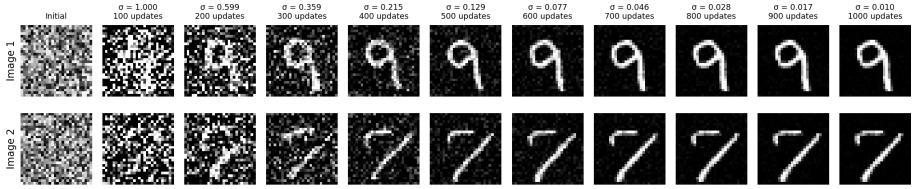
$\mu_1 = (-5, -5)$, $\mu_2 = (5, 5)$ Langevin initialization: $X_0 \sim \mathcal{N}(\mu_1, 0.3^2 I_2)$



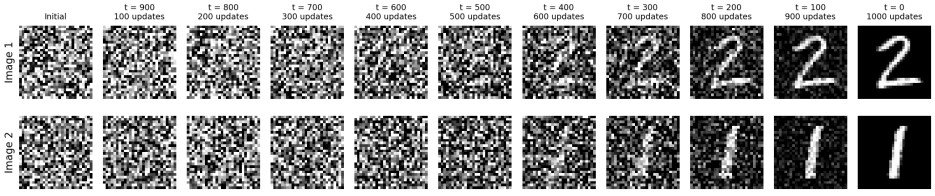
Open animation in browser

Experiment on the MNIST data

Generated by SMLD



Generated by DDPM



Open animation in browser

DDPM in continuous time

Rescale the DDPM step. For $t \in [0, 1]$, use noise variance $\beta(t)\Delta t$:

$$X_{t+\Delta t} = \sqrt{1 - \beta(t)\Delta t}\, X_t + \sqrt{\beta(t)\Delta t}\, \epsilon,$$

where $\epsilon \sim \mathcal{N}(0, I_D)$ is independent at each step.

By Taylor's expansion, $\sqrt{1-u} \approx 1 - u/2$,

$$X_{t+\Delta t} - X_t \approx -\frac{1}{2}\beta(t) X_t\, \Delta t + \sqrt{\beta(t)}\, \underbrace{\sqrt{\Delta t}\, \epsilon}_{\Delta W_t}.$$

W_t is D-dimensional Brownian motion, with $\Delta W_t \sim \mathcal{N}(0, \Delta t\, I_D)$.

As $\Delta t \to 0$: the variance-preserving (VP) SDE

$$\mathrm{d}X_t = \underbrace{-\frac{1}{2}\beta(t) X_t\, \mathrm{d}t}_{\text{signal decay}} + \underbrace{\sqrt{\beta(t)}\, \mathrm{d}W_t}_{\text{noise injection}}.$$

SMLD in continuous time

Interpolate the noise schedule. Let $\sigma^2(t)$ be differentiable and increasing, with $\sigma(0) = 0$. Use independent Gaussian increments:

$$X_{t+\Delta t} - X_t = \sqrt{\sigma^2(t+\Delta t) - \sigma^2(t)}\,\epsilon, \qquad \epsilon \sim \mathcal{N}(0, I_D)$$

$$= \sqrt{\frac{\sigma^2(t+\Delta t) - \sigma^2(t)}{\Delta t}}\,\underbrace{\sqrt{\Delta t}\,\epsilon}_{\Delta W_t}.$$

As $\Delta t \to 0$: the variance-exploding (VE) SDE

$$\mathrm{d}X_t = \underbrace{\sqrt{\frac{\mathrm{d}\sigma^2(t)}{\mathrm{d}t}}}_{\text{noise injection}}\,\mathrm{d}W_t\,.$$

Its conditional distribution matches the SMLD perturbation:

$$X_t \mid X_0 \sim \mathcal{N}\Big(X_0, \sigma^2(t) I_D\Big)\,.$$

A general SDE framework

Forward SDE: data to noise $(t : 0 \to 1)$. With a scalar noise scale $g(t)$,

$$\mathrm{d}X_t = \underbrace{f(X_t, t)}_{\text{drift}}\,\mathrm{d}t + \underbrace{g(t)}_{\text{noise scale}}\,\mathrm{d}W_t.$$

- **DDPM** is a discretization of the **VP** choice: $f(x,t) = -\frac{1}{2}\beta(t)x,\ g(t) = \sqrt{\beta(t)}$.
- **SMLD** is a discretization of the **VE** choice: $f(x,t) = 0,\ g(t) = \sqrt{d(\sigma^2(t))/dt}$.

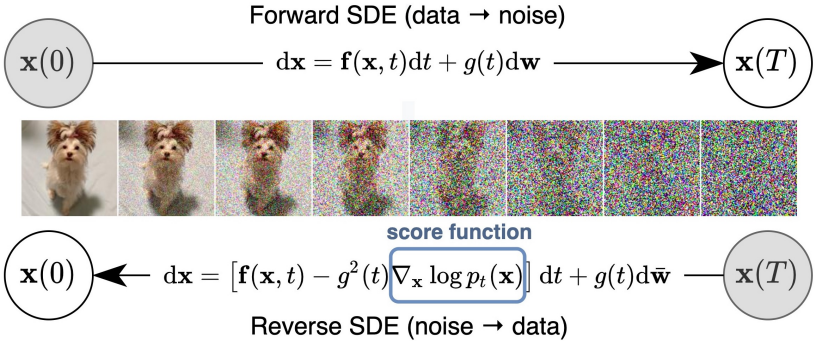
Reverse SDE: noise to data $(t : 1 \to 0)$. Let q_t denote the density of X_t.

$$\mathrm{d}X_t = \left[f(X_t, t) - g(t)^2 \underbrace{\nabla_x \log q_t(X_t)}_{\text{score}} \right] \mathrm{d}t + g(t)\,\mathrm{d}\overline{W}_t.$$

Here $\mathrm{d}t < 0$, and $\overline{W}_t$ is Brownian motion in reverse time.

Learn the score $s_\theta(x_t, t) \approx \nabla_x \log q_t(x_t)$, then simulate backward from noise.

A general SDE framework



- For a unified SDE perspective, see Song et al. (ICLR, 2021).
- We will develop the mathematical details in the next lecture.

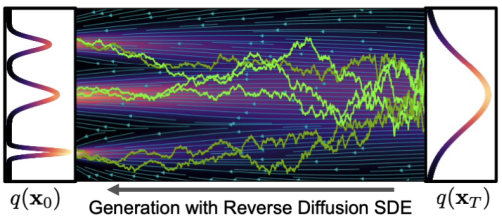
The probability-flow ODE

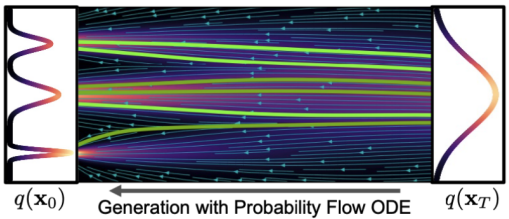
Idea: Move noise samples toward data using a deterministic velocity field.

$$\boxed{\frac{\mathrm{d}X_t}{\mathrm{d}t} = f(X_t, t) - \frac{1}{2} g(t)^2 \nabla_x \log q_t(X_t).}$$

For generation, integrate backward from $t = 1$ to $t = 0$.

Why it works: The score correction makes the ODE move probability mass in the same way as the SDE. With the true score and the same starting distribution, both have the same marginals q_t.





References

- Song, Y. and Ermon, S. (2019). *Generative Modeling by Estimating Gradients of the Data Distribution.* NeurIPS. Paper
- Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. (2021). *Score-Based Generative Modeling through Stochastic Differential Equations.* ICLR. Paper
- Vincent, P. (2011). *A Connection Between Score Matching and Denoising Autoencoders.* Neural Computation, 23:1661–1674. Paper
- Song, Y. (2021). *Generative Modeling by Estimating Gradients of the Data Distribution.* Blog post. Blog
- Ermon, S. *CS236: Deep Generative Models.* Stanford University. Course materials